Design Report

Model Extraction — Stealing a Classifier via a Black-Box HTTP Prediction API

Shatabdi Dutta Chowdhury, 2105123 *Tasnimzaman Tanmi, 2105147*

September 9, 2026

1 Overview of the Project Idea

Machine-learning-as-a-service (MLaaS) platforms expose trained models through query-only prediction APIs: a client submits an input and receives a prediction (a class label, and often a confidence/probability vector) in return, while the model's architecture, parameters, and training data remain hidden. A **model extraction attack** abuses this query interface to build a local *substitute* model that reproduces the victim's behavior, without ever seeing its internals [1, 2, 3]. This directly threatens the confidentiality of the deployed model (its intellectual property and training-data privacy) and can be a stepping stone toward other attacks such as adversarial-example transfer or evasion.

1.1 Why this matters

A successful extraction lets an adversary who has no internal access to a paid or proprietary API:

- obtain a functionally equivalent (or near-equivalent) local copy of the model and avoid further per-query charges;
- probe the substitute model *offline* to craft further attacks (e.g., evasion or adversarial examples) against the real victim;
- in some cases (kernel/linear models), leak information about the victim's private training data through the recovered parameters.

1.2 Project Goal

We plan to implement and evaluate model extraction end-to-end against a *real, network-reachable* victim, not merely an in-process function call. Concretely:

1. Train two victim classifiers on real MNIST images and serve them behind a genuine Flask HTTP prediction API (two model types because each attack targets a different output structure – see Section 4).
2. Implement an **equation-solving attack** (Tramèr et al. [1], Sec. 4.1) against the linear softmax victim, recovering its weight matrix W and bias b *exactly* using linear algebra on the log-odds of returned probabilities.

3. Implement a **knockoff-net-style attack** (Orekondy et al. [2]; to be evaluated using the accuracy/fidelity framework of Jagielski et al. [3]) against the CNN victim, training our own substitute CNN purely from (image, pseudo-label) pairs returned by the API.
4. Design and evaluate **API-layer defenses** that the victim can deploy to blunt these attacks, and quantify the resulting utility/security trade-off (bonus component).

The attacker process is entirely separate from the victim process and communicates with it *only* over HTTP – it never imports the victim's model code, mirroring the black-box threat model of all three reference papers.

2 Network Topology, Components, and Timing Diagrams

2.1 System / Network Topology

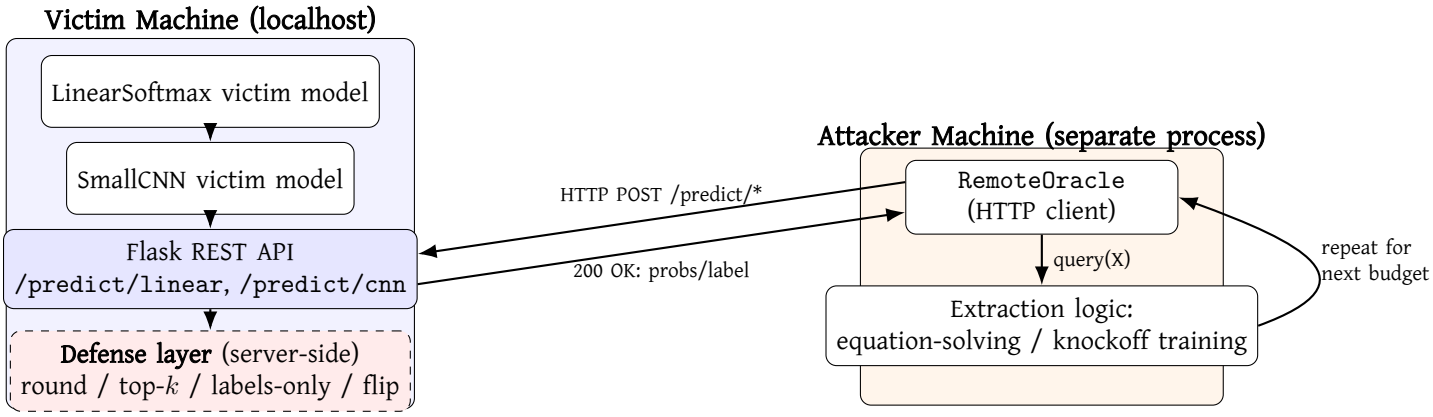


Figure 1: System topology. The attacker and victim run as two independent processes connected only by an HTTP/TCP channel (loopback for the local demo; a real deployment would place them on separate hosts across the Internet). The attacker never has access to the victim's weights, training data, or source code – its only interface is the JSON request/response pair defined in Section 3.

Components :

- **Victim process**: Flask HTTP server hosting two PyTorch models (`LinearSoftmax`, `SmallCNN`) trained on real MNIST. Two server instances are used: an *undefended* API (port 5000) and a *defended* API (port 5001) whose output-perturbation policy is fixed by the operator at start-up.
- **Attacker process**: a Python client (`RemoteOracle`) that issues batched `POST` requests over TCP/HTTP and exposes a minimal `query(X)` interface to the attack algorithms (`attack_equation_solving.py`, `attack_knockoff.py`). The attack code is *transport-agnostic*: it only calls `oracle.query(...)` and never imports victim code.
- **Transport**: standard HTTP/1.1 over TCP (loopback, `127.0.0.1`); JSON request/response bodies.

2.2 Timing Diagram: Normal (Honest-User) Query

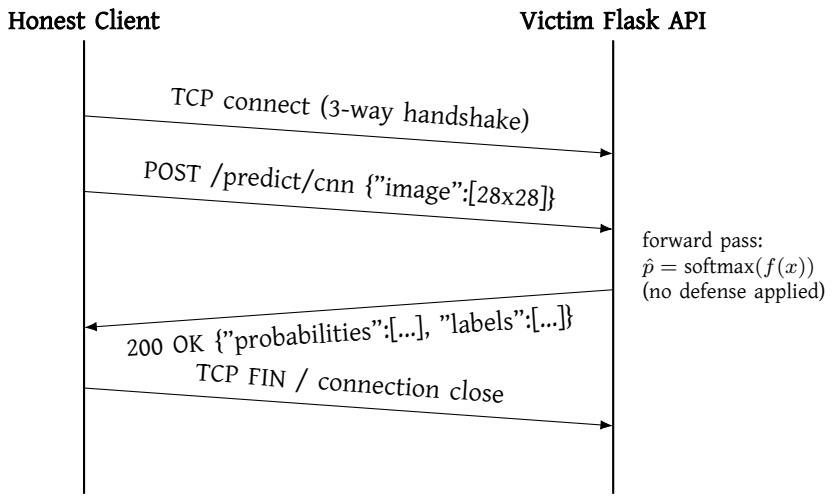


Figure 2: Timing diagram of the *original protocol*: a single honest prediction request receives the true softmax probability vector, unmodified.

2.3 Timing Diagram: Equation-Solving Attack (Tramèr et al.)

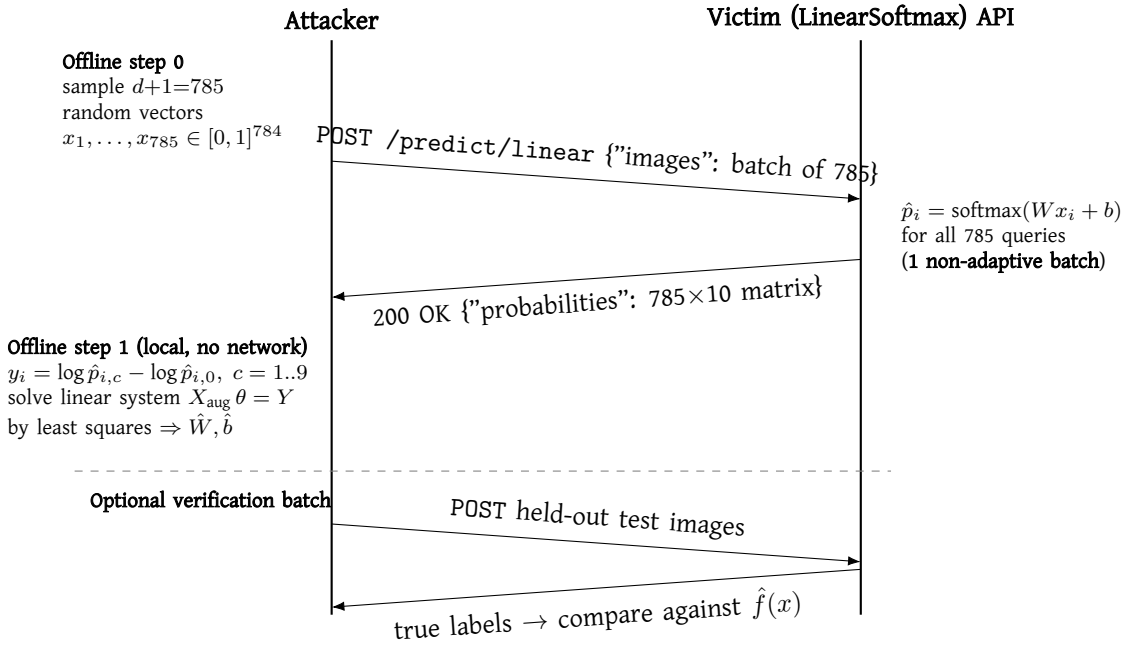


Figure 3: Timing / strategy diagram for the equation-solving attack. Because the victim's output is a *log-linear* function of the input, a single non-adaptive batch of $d+1$ queries is provably sufficient to set up a fully-determined linear system; the attack requires no further interaction with the victim after that batch.

2.4 Timing Diagram: Knockoff-Net Attack (Orekondy / Jagielski)

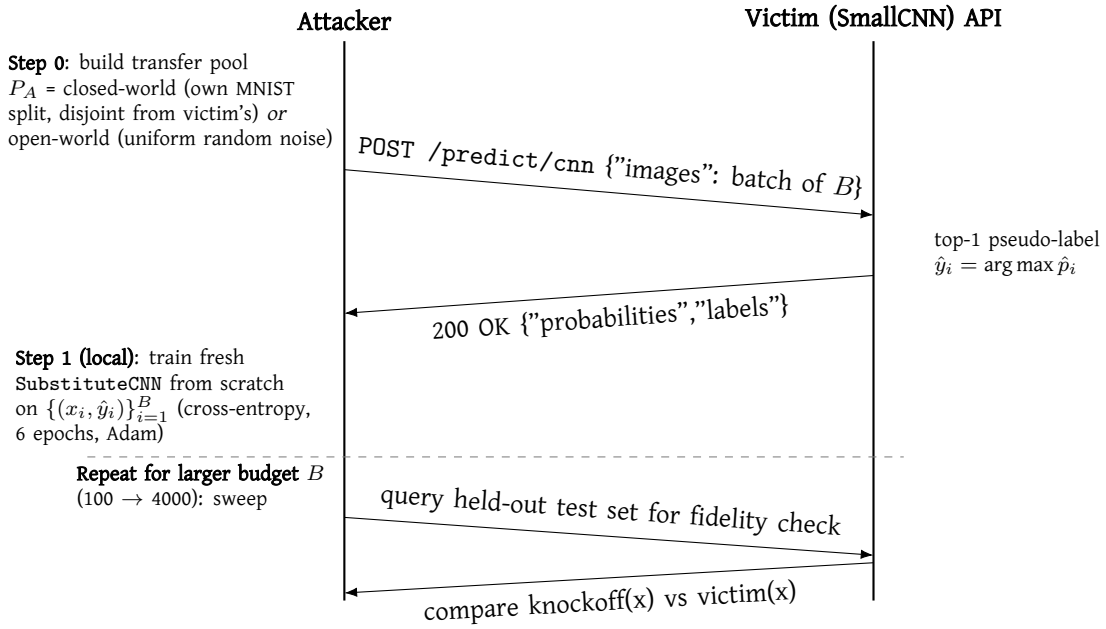


Figure 4: Timing / strategy diagram for the knockoff attack. Unlike equation-solving, this attack is purely label-driven, makes *no* assumption about the victim's architecture, and its accuracy/fidelity improve monotonically with the query budget B (swept at $B \in \{100, 300, 800, 2000, 4000\}$).

3 Packet / Frame (API Request-Response) Structure

Because the "network" here is an application-layer HTTP API rather than a raw link/network-layer protocol, the equivalent of a "packet/frame" for this project is the well-defined **JSON request/response contract** of the prediction endpoint. All attack and defense logic hinges on the exact fields available in this contract.

3.1 Endpoint summary

Endpoint	Purpose
`POST /predict/linear`	Query the `LinearSoftmax` victim (equation-solving target)
`POST /predict/cnn`	Query the `SmallCNN` victim (knockoff target)
`GET /health`	Liveness check + which defense policy is active

3.2 Request frame

```
POST /predict/cnn HTTP/1.1
Host: localhost:5001
Content-Type: application/json
```

```
{
  "images": [ [ /* 28 x 28 floats in [0,1] */ ], ... ], // batched
  // or, single-image form:
  "image": [ /* 28 x 28 floats in [0,1] */ ],

  // OPTIONAL, only honoured by the defended server:
  "defense": {
      "mode": "round" | "topk" | "label_only" | "flip" | "none",
      "decimals": 2, // for "round"
      "k": 3, // for "topk"
      "epsilon": 0.3 // for "flip"
  }
}
```

3.3 Response frame

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "probabilities": [ [p0, p1, ..., p9], ... ], // N x 10, post-defense
  "labels": [ argmax class per image, ... ],
  "count": N,
  "defense": "none" | "round(2d)" | "top-3" | "label_only" | "flip(eps=0.3)"
}
```

3.4 Field-level notes relevant to the attack design

- **Batching**: queries are sent in batches (default 256 images per HTTP request) purely for network efficiency, but the *query budget* is still counted per image, exactly as in Tramèr et al. [1] and Orekondy et al. [2] – one image = one unit of adversarial "cost."
- **Confidence values** (`probabilities`) are the field the equation-solving attack depends on: it needs the *exact* soft-max probability vector to form the linear log-odds system (Section 4.3). Any perturbation of this field (the `round`, `topk`, or `label_only` defenses) directly corrupts that system.
- **Top-1 label** (`labels[0]`) is the *only* field the knockoff attack depends on – it never reads `probabilities`. This is why label-preserving defenses (round/top-k/label-only) fail to stop it, while the label-perturbing `flip` defense does hurt it (Section 4.5).
- **Server-side defense injection point**: the `defense` spec is applied to the model's raw `probs` tensor *inside the Flask route handler*, immediately before serialization to JSON – i.e., at the exact network boundary a real cloud ML API (e.g. a commercial vision/prediction service) would use, not inside the model itself.

4 Implementation Plan

4.1 Why two different victim models

The two reference attacks target structurally different output functions, so two victims are trained:

- **`LinearSoftmax`** ($784 \to 10$, a single linear layer + softmax): its output is *log-linear* in the input, which is the exact precondition the equation-solving attack of Tramèr et al. [1] requires.
- **`SmallCNN`** (2 conv layers + 2 FC layers): a non-linear model with an architecture the attacker is assumed *not* to know, matching the weaker-adversary assumptions of Orekondy et al. [2] (knockoff nets) and the accuracy/fidelity evaluation framework of Jagielski et al. [3].

Both will be trained on real MNIST (60,000/10,000 handwritten digit images, 28×28 grayscale, 10 classes), subsampled to 8,000 train / 2,000 test images to keep CPU training time practical. We expect clean test accuracy in the neighbourhood of standard baselines for these architectures on MNIST – roughly high-80s to low-90s% for the single-layer linear softmax model, and mid-90s% or higher for the small CNN.

4.2 Planned Components and Responsibilities

Rather than committing to file names before implementation begins, the codebase is planned as three logical groups of components, each with a clearly scoped responsibility. This keeps the victim and attacker sides strictly separated (see the isolation guarantee below) while leaving the exact module/file layout to be finalized during implementation.

Victim-side components
Load real MNIST data (with a hand-written IDX-parser fallback in case a library loader is unavailable)
Shared plain-PyTorch train/eval loop, reused by both the victim and the attacker's substitute model
`LinearSoftmax` and `SmallCNN` architecture definitions (kept secret from the attacker side)
Training routine that fits and saves both victim models to disk
Output-perturbation functions for the four candidate defenses: round / top-k / label-only / label-flip
Flask application factory: loads a model, applies the active defense policy, and serves JSON predictions
Undefended API entry point (port 5000)
Defended API entry point, with defense policy fixed by a start-up CLI argument (port 5001)
Attacker-side components
HTTP client wrapper – the *only* channel through which the attacker may interact with the victim
Equation-solving attack logic (Tramèr et al.)
Attacker's own, independently declared substitute CNN architecture
Knockoff-style learning attack logic (Orekondy / Jagielski)
Experiment / bonus components
Full budget/defense sweep driver, producing a results log and summary plots
Bonus defense analysis: quantifies *why* each defense does or does not work

Isolation guarantee: the attacker-side components will never import anything from the victim-side components. The only coupling between the two sides is the HTTP contract in Section 3, enforced at the code level so the implementation cannot accidentally "cheat" by peeking at victim internals.

4.3 Attack 1 – Equation Solving (design)

For the linear victim, $f_i(x) = \text{softmax}(Wx+b)_i$. Taking log-odds relative to class 0 eliminates the softmax normalizer and yields, for every query x and every class $c \neq 0$:

$$\log \frac{f_c(x)}{f_0(x)} \;=\; (w_c - w_0) \cdot x + (b_c - b_0),$$

which is **linear** in the unknown parameters. With input dimension $d = 784$ (a flattened 28×28 image), $d+1 = 785$ unknowns per class must be recovered, so 785 linearly-independent query points give a fully-determined system. Design steps:

1. Draw 785 query images with i.i.d. pixels $\sim \text{Uniform}(0, 1)$ (any real-valued input works; digit-like structure is *not* required) and send them as one batched HTTP request.
2. From the returned probability matrix, compute the log-odds matrix $Y \in \mathbb{R}^{785\times 9}$.
3. Solve $X_{\text{aug}}\,\theta = Y$ by least squares (`numpy.linalg.lstsq`) to recover $\hat{W}, \hat{b}$ – no gradient descent, no training loop.
4. Evaluate the extracted model's *accuracy* (vs. ground-truth labels) and *fidelity* (agreement with the victim's own predictions) on a held-out test batch.

This attack is **non-adaptive** (a single query batch, chosen before seeing any response) and is expected to be essentially *exact*, since the victim genuinely is log-linear.

4.4 Attack 2 – Knockoff-Net (design)

For the CNN victim, no closed-form relationship exists between input and output, so a learning-based strategy is used instead:

1. **Transfer-set construction.** Two query distributions P_A are studied, following Orekondy et al.'s closed-/open-world split:
   - *Closed-world*: real MNIST images from a pool disjoint from the victim's own training subsample (different random seed) – the attacker has *pixels*, never the victim's labels.
   - *Open-world*: pure uniform-random noise images – a maximally weak/generic adversary with no domain knowledge.
2. **Pseudo-labeling.** Each transfer image is queried once; only the returned top-1 label is kept (`argmax` of `probabilities`).
3. **Substitute training.** A freshly-initialized `SubstituteCNN` (declared independently on the attacker side, *not* imported from the victim) is trained from scratch on $\{(x_i, \hat{y}_i)\}$ with cross-entropy loss, Adam, 6 epochs.
4. **Budget sweep.** Steps 1–3 are repeated at budgets $B \in \{100, 300, 800, 2000, 4000\}$ to reproduce an Orekondy-style accuracy/fidelity-vs-budget curve (cf. Jagielski et al.'s accuracy/fidelity taxonomy).

4.5 Bonus – API-layer Defense Suite

Four defenses are planned, all to be applied *server-side* at the API boundary (Section 3), never inside the model itself:

Defense	Mechanism	Reference
`round`	Round each p_i to k decimals, renormalize	Tramèr et al.
`topk`	Zero out all but the top-k probabilities, renormalize	Tramèr et al.
`label_only`	Return a one-hot vector for the top-1 label only	Orekondy et al.
`flip`	Return a *deterministically-seeded* wrong label with probability ε (randomized response on the output label, keyed by a hash of the input pixels)	Open problem named in all three papers, but not implemented by any

The first three defenses *preserve the top-1 label* and are therefore expected to cripple equation-solving (which needs exact confidences) while leaving the knockoff attack almost

untouched (it only ever reads the label). The `flip` defense is the only one expected to directly damage the knockoff attacker, because it corrupts the very pseudo-labels the substitute model is trained on – at a direct, quantifiable cost to honest-user accuracy ($\approx \varepsilon$).

5 Expected Outcome of a Successful Attack

5.1 Equation-Solving vs. LinearSoftmax

Since the `LinearSoftmax` victim's output is exactly log-linear in the input, the equation-solving attack has a closed-form correctness guarantee: a single non-adaptive batch of $d + 1 = 785$ linearly-independent queries fully determines the linear system, so we expect the extracted weights $(\hat{W}, \hat{b})$ to match the victim to numerical precision, and the extracted model's fidelity (agreement with the victim) to approach 100% on held-out test images – consistent with the near-perfect extraction Tramèr et al. [1] report for logistic models on production APIs (Table 1 of their paper). Because the required query count is fixed at $d + 1$ and does not depend on how much data the victim used for training, we also expect the attack's cost to be *independent* of the victim's training-set size.

5.2 Knockoff Attack vs. SmallCNN

Following the closed-/open-world results reported by Orekondy et al. [2], we expect substitute accuracy and fidelity to **increase with query budget** in the closed-world setting (real, task-relevant transfer images drawn from a disjoint MNIST split), approaching a sizeable fraction of the victim's own accuracy once the budget reaches the low thousands of queries. In the open-world setting (pure random-noise transfer images) we instead expect the substitute to **stay close to chance level** ($\approx 10\%$, i.e. $1/10$ classes) regardless of budget, since pseudo-labels produced on structureless noise carry little information about the true digit decision boundary. This would directly illustrate Orekondy et al.'s central finding that *query-distribution realism*, not merely query count, is what drives extraction success.

5.3 Defense Effectiveness (Bonus)

Based on the countermeasure results reported by Tramèr et al. [1] (Section 7, rounding/truncation experiments) and the discussion of label-only APIs in Orekondy et al. [2], we expect the following qualitative pattern:

- Confidence-hiding defenses (`round`, `topk`, `label_only`) should **sharply degrade** equation-solving, since that attack's linear system is built directly from precise probability values – rounding or truncating them corrupts the log-odds targets it solves for.
- The same three defenses should leave the knockoff attack's accuracy **largely unaffected**, since that attack only ever reads the top-1 predicted label, which all three defenses deliberately preserve.
- The label-`flip` defense is expected to be the *only* member of the suite that meaningfully damages the knockoff attacker, because it is the only defense that corrupts the pseudo-labels themselves – at a direct, quantifiable cost to honest-user accuracy of roughly ε (the flip probability).

This would make explicit the utility/security trade-off that Tramèr, Orekondy, and Jagielski all name as an important open problem but do not themselves quantify – our planned budget/defense sweep (`attacker/run_experiment.py`) is designed specifically to produce that quantification once implementation is complete.

6 Defense Idea (Countermeasure Design)

Beyond simply measuring the four defenses above, the design rationale for *why* each one is placed exactly where it is:

- **Placement at the API boundary, not in the model.** All defenses operate on the already-computed probability tensor, inside the Flask route handler, immediately before JSON serialization (`victim/server_core.py`). This matches how a real commercial prediction API would deploy a countermeasure – without retraining or modifying the underlying model.
- **Confidence-hiding defenses** directly target the *information* equation-solving needs (continuous, high-precision probabilities) while deliberately preserving what honest applications usually need (the top-1 label), so they are cheap, easy-to-deploy countermeasures against *that specific attack family.*
- **The label-flip defense** is designed to be **deterministic per input** (seeded by a keyed hash of the image pixels plus a server secret) specifically so that an attacker *cannot average the injected noise away* by re-querying the same point multiple times – a design detail directly motivated by the randomized-response literature and the open problem named (but not solved) in Tramèr et al. [1].
- **Quantified trade-off.** Because `flip` necessarily also corrupts a fraction ε of honest predictions, the defense is evaluated on *both* axes (attacker fidelity *and* honest-user accuracy) rather than attacker success alone, so that a deploying operator can pick ε according to their own risk/utility tolerance.

Justification: Why This Design Should Work

- **Equation-solving is exact for a log-linear model, given linearly-independent queries.** A linear system with $d+1$ unknowns per class is fully determined by $d+1$ linearly-independent query points; querying random points in $[0,1]^d$ makes linear independence overwhelmingly likely (though not guaranteed), so the least-squares solve is expected to recover (W, b) up to floating-point precision – this is a closed-form derivation, not a heuristic, but at the minimum query budget $n_{\text{queries}} = d+1$ the resulting linear system is exactly-determined rather than over-determined, so unlike Section 4.3's optional verification step, recovery is not additionally robust to ill-conditioned query batches. We therefore also report results at $n_{\text{queries}} > d+1$ (over-determined, least-squares-smoothed) as a robustness check.
- **Knockoff attacks require no assumption about victim internals**, only the ability to query and observe a label – exactly the minimal capability every prediction API must expose to be useful at all, which is why this attack family is considered practically unavoidable for CNN-class victims without output-perturbation defenses.

- **The two attacks are complementary by design**: one exploits the victim's exact mathematical structure (fast, exact, but only works on log-linear models); the other exploits nothing but query access (slower, approximate, but architecture-agnostic) – together they cover both ends of the accuracy/fidelity taxonomy of Jagielski et al. [3].
- **The HTTP/Flask deployment is a faithful stand-in for a real MLaaS system**: separate attacker/victim processes, a fixed JSON API contract, and server-side-only defense placement reproduce the exact threat model (black-box query access, no source/weight access) used by all three reference papers, so results obtained here are expected to generalize qualitatively to real cloud prediction APIs.

References

[1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing Machine Learning Models via Prediction APIs," *25th USENIX Security Symposium*, 2016.

[2] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff Nets: Stealing Functionality of Black-Box Models," *IEEE CVPR*, 2019.

[3] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, "High Accuracy and High Fidelity Extraction of Neural Networks," *29th USENIX Security Symposium*, 2020.